

Experiment No. 5
Use SMOTE technique to generate synthetic data on the given dataset and evaluate the performance
Date of Performance: 24/02/2025
Date of Submission:24/02/2025

Aim: Use SMOTE technique to generate synthetic data on the given dataset and evaluate the performance.

Objective: Able to work with imbalance in the data and identify the impact of balancing the dataset.

Theory:

SMOTE is a technique in machine learning for dealing with issues that arise when working with an unbalanced data set. In practice, unbalanced data sets are common and most ML algorithms are highly prone to unbalanced data so we need to improve their performance by using techniques like SMOTE

Synthetic Minority Over-sampling TEchnique, or SMOTE for short, is a preprocessing technique used to address a class imbalance in a dataset.

In the real world, oftentimes we end up trying to train a model on a dataset with very few examples of a given class (e.g. rare disease diagnosis, manufacturing defects, fraudulent transactions) which results in poor performance. Due to the nature of the data (occurrences are so rare), it's not always realistic to go out and acquire more. One way of solving this issue is to under-sample the majority class. That is to say, we would exclude rows corresponding to the majority class such that there are roughly the same amount of rows for both the majority and minority classes. However, in doing so, we lose out on a lot of data that could be used to train our model thus improving its accuracy (e.g. higher bias). Another other option is to over-sample the minority class. In other words, we randomly duplicate observations of the minority class. The problem with this approach is that it leads to overfitting because the model learns from the same examples. This is where SMOTE comes in. At a high level, the SMOTE algorithm can be described as follows:

1. Take difference between a sample and its nearest neighbour
2. Multiply the difference by a random number between 0 and 1
3. Add this difference to the sample to generate a new synthetic example in feature space
4. Continue on with next nearest neighbour up to user-defined number

Code:

```
import numpy as np

import pandas as pd

from imblearn.over_sampling import SMOTE

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import classification_report, accuracy_score

# Sample dataset creation

data = {

    'feature1': np.random.rand(1000),

    'feature2': np.random.rand(1000),

    'feature3': np.random.rand(1000),

    'target': np.concatenate((np.zeros(900), np.ones(100))) # Imbalanced dataset

}

df = pd.DataFrame(data)

# Ensure target column exists

if 'target' not in df.columns:

    raise ValueError("Target column 'target' not found in dataset")

# Define features and target
```

```
X = df.drop(columns=['target']) # Replace 'target' with actual target column name

y = df['target']


# Handle potential NaN values in dataset

X.fillna(X.mean(), inplace=True)


# Split data into train and test sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)


# Check class distribution before SMOTE

print("Class distribution before SMOTE:")

print(y_train.value_counts())


# Apply SMOTE with error handling

try:

    smote = SMOTE(random_state=42)

    X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)

except ValueError as e:

    raise ValueError("Error applying SMOTE: " + str(e))


# Check class distribution after SMOTE

print("Class distribution after SMOTE:")

print(y_train_smote.value_counts())
```

Train a classifier before applying SMOTE

clf_before = RandomForestClassifier(random_state=42, n_jobs=-1)

clf_before.fit(X_train, y_train)

y_pred_before = clf_before.predict(X_test)

Train a classifier after applying SMOTE

clf_after = RandomForestClassifier(random_state=42, n_jobs=-1)

clf_after.fit(X_train_smote, y_train_smote)

y_pred_after = clf_after.predict(X_test)

Evaluate models

print("Performance before applying SMOTE:")

print(classification_report(y_test, y_pred_before))

print("Accuracy:", accuracy_score(y_test, y_pred_before))

print("Performance after applying SMOTE:")

print(classification_report(y_test, y_pred_after))

print("Accuracy:", accuracy_score(y_test, y_pred_after))

Output:

Run	Just now
Class distribution before SMOTE:	module:33
target	module:34
0.0 720	
1.0 80	
Name: count, dtype: int64	
Class distribution after SMOTE:	module:44
target	module:45
0.0 720	
1.0 720	
Name: count, dtype: int64	
Performance before applying SMOTE:	module:58
precision recall f1-score support	module:59
0.0 0.90 0.99 0.94 180	
1.0 0.00 0.00 0.00 20	
accuracy 0.90 200	
macro avg 0.45 0.50 0.47 200	
weighted avg 0.81 0.90 0.85 200	
Accuracy: 0.895	module:60
Performance after applying SMOTE:	module:62
precision recall f1-score support	module:63
0.0 0.89 0.79 0.84 180	
1.0 0.07 0.15 0.10 20	
accuracy 0.72 200	
macro avg 0.48 0.47 0.47 200	
weighted avg 0.81 0.72 0.76 200	
Accuracy: 0.725	module:64

Conclusion: Applying SMOTE successfully balanced the dataset, leading to improved model performance on the minority class. Before SMOTE, the classifier struggled with class imbalance, likely favoring the majority class. After SMOTE, the model demonstrated improved recall for the minority class, reducing bias. However, it is important to ensure that synthetic data does not introduce noise, which could affect overall accuracy.